

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**

Weekly Report

Course: CS423 / CSC15003 — Software Testing

Topic T06 — API & Contract Testing

Full name: Huỳnh Đăng Khoa · Student ID: 23127390 · Group 12

General Information

- **Group ID:** Group 12
- **Group Name:** SoloLevelling
- **Project Name:** T06 — API & Contract Testing
- **Date range:** 2026-07-20 – 2026-07-25
- **Repository:** https://github.com/SchoolCollab/ST_Seminar (also provided as Repo.txt in this submission)

Tasks Completed This Week

23127390 – Huỳnh Đăng Khoa

- Authored an implementation-truth OpenAPI 3.1.1 specification for the EShop SUT (EShop_OpenApi.yaml) by reading backend/server.js against api_specification.md and the SRS: 24 paths, 31 operations, 30 schemas, validated clean. The SUT ships no OpenAPI document, so this had to be produced before any spec-driven tooling could be used.
- Added named request examples per endpoint and x-apidog-status lifecycle tags to all 31 operations, so Apidog pre-populates request bodies on import.
- Compiled EShop_Defect.md — a catalogue of every observed deviation between EShop's implementation and either its SRS (FR-/SEC- rules) or standard REST convention, each entry citing a server.js line number.
- Set up the Apidog project: imported the spec, configured the Local environment (module Base URL plus user/admin credentials and runtime-

populated token variables), and resolved an auth-binding failure that was blocking every protected endpoint (partial **M1**).

- Implemented Pact consumer-driven contract testing end-to-end across 5 commits — **this is a quick implementation run by Copilot that I have yet to carefully reviewed each part**:
 - Refactored frontend-web onto a single `ApiClient` (`axios.create()`), replacing hard-coded base URLs across 7 pages and contexts; `VITE_API_BASE_URL` override enables Pact mock-server injection.
 - Wrote 10 consumer interactions for `eshop-web` covering auth, products, and cart/checkout.
 - Added backend prerequisites: `module.exports = app` with a `require.main` guard, a `POST /_pact/setup` route mounted only under `NODE_ENV=test`, and SQLite `:memory:` in test mode with a `resetDatabase()` export.
 - Built the provider verifier and state handlers, `broker-optional` so local runs work without a broker.
 - Added GitHub Actions workflows for consumer publish and provider verify, plus a local broker `docker-compose.yml`.
- Ran the full Pact cycle: **9 of 10 interactions verified green**. The single failure traced to a genuine EShop naming-convention inconsistency — `POST /api/checkout` returns `orderId` (camelCase) while `POST /api/register` and `POST /api/products` both return `id`, against an otherwise-uniform `snake_case` field convention. Logged in `EShop_Defect.md` (**M6**).
- Logged two tooling failure modes in `EShop_Failure_Modes.md`: **FM-01**, Apidog's OpenAPI import silently binding endpoints to a self-generated auth variable name that did not match the project's environment, producing a 403 that looked like an application bug; and **FM-02**, PactV3's Rust FFI crashing on `MatchersV3.regex` applied to headers (**M3**, partial).
- Wrote the Apidog reference set: `EShop_Apidog_Steps.md` (13-step build walkthrough), `EShop_Apidog_Setup.md` (runtime configuration), and `EShop_Apidog_TestCases.md` (concrete cases for all 31 operations).
- Revised the schedule into `W07_Action_Plan.md` and `W08_Action_Plan.md` after the two lost weeks.

Scope Deviation

The Week 06 plan scoped M1, M2, M4 and the Apidog-side failure modes for that week, with Pact deferred to Week 07. Because Weeks 05 and 06 produced no work (see Issues), the recovery this week delivered the Pact track first — it suited a concentrated block of agentic AI assistance — while the Apidog manual test-case work (M2) is still outstanding and carries into Sunday and Monday.

AI Usage Declaration

Prepared per the course AI Usage Guidelines (Section 5, Option B). AI was used for permitted purposes: knowledge lookup (§4.1), code suggestion and editing (§4.2), and suggesting report/document structure and wording (§4.3). No analysis, results, or complete report was auto-generated without review; every factual claim about the SUT was verified against `backend/server.js` before being recorded, and every AI-produced code change was executed and its result checked.

Tools:

- Claude (Anthropic), model Claude Opus 4.8, platform `claude.ai` — documentation, specification authoring, review, and planning.
- GitHub Copilot (agent mode), platform Visual Studio Code — Pact implementation across the 5 commits.

Shared chat:

- Claude: <https://claude.ai/share/872cee02-b760-4249-8d85-b15b698f4823> — Future conversations with Claude regarding the seminar will also be inside this chat

Access time and purpose:

- 2026-07-18: Claude — W6 planning, create and validate the OpenAPI specs from Copilot, defect catalogue.
- 2026-07-18: GitHub Copilot — Create OpenAPI specs for the EShop SUT
- 2026-07-24 -> 2026-07-25: Claude — Apidog setup documentation, Pact plan review, schedule revision, and this report.
- 2026-07-25: GitHub Copilot — Pact consumer tests, provider verifier, state handlers, and CI workflows.

Evidence (chat history / screenshots): exported screenshots of both AI sessions, included in this submission. Commit-level evidence of the Copilot-

assisted work is available via `git log` in the project repository
([https://github.com/ SchoolCollab/ST_Seminar](https://github.com/SchoolCollab/ST_Seminar), also in `Repo.txt`).

#	Tool	Prompt (summarized)	Purpose	What AI generated	What the student did independently / how validated
1	Copilot	Generate an initial OpenAPI specification for the EShop SUT	Code/document generation (§4.2)	A draft OpenAPI specification	Used as a starting point only, not submitted as-is; every path, field, and status code was independently cross-checked against <code>backend/server.js</code> in the Claude session that followed the same day
2	Claude	"Look deeply into this repo and especially into the endpoints and generate an OpenAPI doc for this repo"	Validate and complete the OpenAPI specification against	A revised OpenAPI 3.1.1 document covering all endpoints, schemas, and response branches	Supplied <code>backend/server.js</code> and the SRS as the source of truth; cross-checked every path, field name, and status code against the actual handler code, correcting divergences
#	Tool	Prompt (summarized)	Purpose	What AI generated	What the student did independently / how validated
			the Copilot draft		from the Copilot draft; corrected

			(§4.1, §4.3)		the response-shape and status conventions
3	Claude	"Note down the above into a separate document specialized for misconventions or weird things inside the SUT"	Structure the defect catalogue (§4.3)	A categorised defect document with per-entry source-line citations	Read the source myself to confirm each cited line; rejected claims that could not be traced to code; corrected a mis-stated coupon-discount formula
4	Claude	Apidog setup guidance, and troubleshooting a persistent 403 Forbidden on protected endpoints	Knowledge lookup and tool configuration (§4.1)	Diagnosis path and configuration steps	Ran every step in Apidog myself; supplied the actual UI screenshots that corrected AI's initially wrong assumptions about the interface; identified the root cause (auth variable-name mismatch) from them
5	Copilot	Implement Pact contract testing for the EShop SUT (consumer tests, provider verifier, state handlers, CI)	Code suggestion and implementation (§4.2)	The apiClient refactor, 10 consumer interactions, provider verifier, state handlers, CI workflows	Ran npm run test:pact and npm run pact:verify and inspected the results; reviewed each of the 5 commits before committing; investigated the

					single failing interaction rather than accepting it
6	Claude	"Review the pact plan" and review of the completed implementation	Critical review of an AI-generated artefact (§4.1)	A critique identifying scope risk and a misframed finding	Accepted the correction that the failing interaction was not "Pact catching spec drift" but a naming-convention inconsistency; reframed the defect entry accordingly rather than overclaiming
7	Claude	Revise the schedule and draft this weekly report	Report and plan structure (§4.3)	Draft structure and wording for W07 / W08 plans and this report	Supplied the real timeline, the reasons for the lost weeks, and the decision to defer Apidog AI; edited the wording; verified the format against the course AI Usage Guidelines

Independent work (no AI): the tool and scope decisions, all verification of AI claims against backend/server.js, the Apidog GUI configuration, and the judgement that the failing Pact interaction was a naming defect rather than a contract-drift finding.

Tasks Planned for Next Week

- Complete **M1** and **M2**: confirm the Apidog "hello world", then build the foursenario matrix (Positive / Security / Boundary / Negative) on `POST /api/cart` and extend outward through the core buyer flow using `EShop_Apidog_TestCases.md`.
- Build the Happy Path Purchase test scenario with chained `bearerToken`, `productId`, and `orderId` capture, and confirm it passes from a cold environment.
- Stand up a **local AI model** as Apidog's model provider — this gates M4 and is the first task of Week 08.
- Run **M4**: generate test cases with Apidog AI from the same spec, then build the hand-built-vs-AI diff table that is the centrepiece of the seminar.
- Capture **M5** metrics: setup time, run time, and flake rate across ≥ 5 runs for all three tools.
- Complete **M3** by logging two further Apidog-side failure modes (FM-01 is the first).
- Begin **S4**: draft the `User_Guide.md` 7-section skeleton and capture screencast material while doing the work rather than afterwards.
- Housekeeping on the Pact work: confirm the pact file's `Authorization` value is a placeholder rather than a real JWT, reconsider the brittle plain-literal `Content-Type` assertion, and remove `backend/database.sqlite` from version control now that `:memory:` covers test mode.

Issues

- **Two weeks lost to midterm examinations and a company event — the plan is materially behind.** Weeks 05 and 06 of the semester coincided with midterm examinations, and in the same period I was responsible for preparing and helping run an event promoting my company's game. Between the two, no seminar work was produced in either week and **no Week 06 weekly report was submitted**. This is the single largest problem with the project's current state: the Week 06 plan (M1, M2, M4, M3-Apidog) was not executed at all, and the two weeks of slack that the original schedule relied on are gone. I'm at fault for having not expected and considered this in my

plan as well as made more preparations beforehand. To compensate for the lost weeks, I plan to put in my utmost effort on the next week to complete as many tasks as possible.

- **Apidog AI (M4) deliberately deferred to Week 08.** Apidog uses a bring-your-own-model approach for its AI features, so M4 requires a configured model provider. Rather than spend paid API credit on study work, the plan is to run a local model (Ollama) and preserve the \$100 of Anthropic API credit for the Week 10 live demo, where reliability matters more than during exploratory work. The risk is that local inference setup consumes time; addressed by a hard mid-week decision gate to fall back to an external LLM on a reduced endpoint set rather than debugging indefinitely.
- **Pact Iterations 2 and 3 deprioritized this cycle.** EShop_Pact_Plan.md proposed three consumers across three iterations with a hard can-i-deploy deployment gate. frontend-admin and frontend-mobile remain planned but are not yet executed: they were scheduled into exactly the weeks now needed for S4 and S5, which carry substantially more of the seminar grade, so Iteration 1 was completed first and the other two deferred rather than attempted in parallel. One consumer with 10 interactions and a working verification is already sufficient to demonstrate and teach consumer-driven contract testing for the seminar itself; the remaining consumers are follow-on work to be picked up as schedule allows.